Análise de Redes: Rhesus Interareal Cortical Network

Ayne Gabriele Figueiredo - GRR20225555
Lilian Miranda Mendes - GRR20222549

1 Introdução

Trabalho final da disciplina CI1054 - Algoritmos em grafos e otimização, no qual foi feito a analise da estrutura da rede cortical inter-regional de um macaco (*Rhesus_interareal.cortical.network_2*), obtida em `http://www.core-nets.org/index.php?action=download`. A análise foi realizada utilizando a linguagem Python e as bibliotecas `NetworkX` e `Matplotlib`.

2 Detalhes do código

O algoritmo para análise foi desenvolvido no **Colab**, e para ser testado é necessário que o arquivo da rede esteja na mesma pasta que o algoritmo. Todos os testes foram feitos no **Colab** por isso está sendo enviado junto deste relatório tanto a versão com extensão .py quanto a versão .ipynb.

3 Metodologia

A análise foi dividida em diferentes partes, sendo elas:

- Componentes conexas
- Anãlise de ciclos
- Centralidade
- Distâncias
- Grau dos vértices
- AGM, clusters e cliques
- Ciclos eulerianos e hamiltonianos

3.1 Componentes conexas

A primeira análise feita foi das componentes conexas, usamos as funções `number_weakly_connected_compo` e `number_strongly_connected_components` da biblioteca **networkx** para classifica-las como fracamente conexas ou fortemente conexas

3.2 Análise de ciclos

Após isso fizemos a análise de ciclos, verificamos se a rede era uma DAG, usando a função `is_directed_acyclic_graph`, além de se havia ciclos de um vértice com ele mesmo, usamos a função `nodes_with_selfloops` para obter essa informação.

Também verificamos se havia ciclos de tamanho 3 usando a função `triadic_census`. Essa função retorna diversos tipos de ciclos, mas para a nossa análise usamos apenas a que verifica um ciclo perfeito entre três vértices(A -> B -> C -> A).

3.3 Centralidade

A próxima análise feita foi a relacionada a centralidade. Fizemos a análise dos três tipos de centralidade vistos em aula:

- **degree_centrality**: Centralidade de Grau
- **betweenness_centrality**: Centralidade de Intermediação
- **closeness_centrality**: Centralidade de Proximidade

3.4 Distâncias

Para fazer a análise de distâncias, verificamos primeiro se a rede era fortemente conexa usando a função `is_strongly_connected`.

Após isso, foram feitas as análises de:

- excentricidade - `eccentricity`
- raio - `radius`
- diâmetro - `diameter`
- centro - `center` (excentricidade == raio)
- periferia - `periphery` (excentricidade == diâmetro)

Sendo essas funções todas da biblioteca **networkx**.

3.5 Grau dos vértices

Após isso também fizemos a análise dos graus dos vértices, verificando os graus de saída e entrada, fazendo também o calculo do maior/menor grau, a média e a frequência. Para essa análise utilizamos as funções `in_degree` e `out_degree`.

3.6 AGM, clusters e cliques

A próxima análise foi em relação a AGM, clusters e cliques.

- **AGM**: Usamos a função `maximum_spanning_tree` para gerar a árvore geradora mínima, além disso colocamos como um dos retornos o némero de arestas que ela usa para conectar todos os vértices.
- **Clusters**: Usamos a função `average_clustering` para descobrir o coeficiente de agrupamento médio.
- **Cliques**: Usamos a função `find_cliques` para calcular a quantidade de cliques da rede, também colocamos como um dos retornos o tamanho da maior clique e os vértices que formam essa clique.

3.7 Ciclos eulerianos e hamiltonianos

A última análise realizada foi em relação a ciclos eulerianos e hamiltonianos.

Para identificar ciclos eulerianos usamos as funções da biblioteca **Networkx** que indicam se há ou não um ciclo:

- `is_eulerian`
- `has_eulerian_path`

E para verificar se havia ciclos hamiltonianos usamos o problema do caixeiro viajente, como visto em aula.

4 Ilustração da rede complexa analizada

Para ilustrar a rede, além da biblioteca **networkx** usamos também a **matplotlib**, para fazer a imagem usamos as seguintes funções:

- `figure`, que configura o tamanho da imagem.
- `spring_layout`, que configura a posição dos vértices.
- `draw_networkx_nodes` e `draw_networkx_edges`, para desenhar os vértices e arestas.
- `draw_networkx_labels`, para escrever o nome dos vértices.
- `title`, para dar nome a imagem.

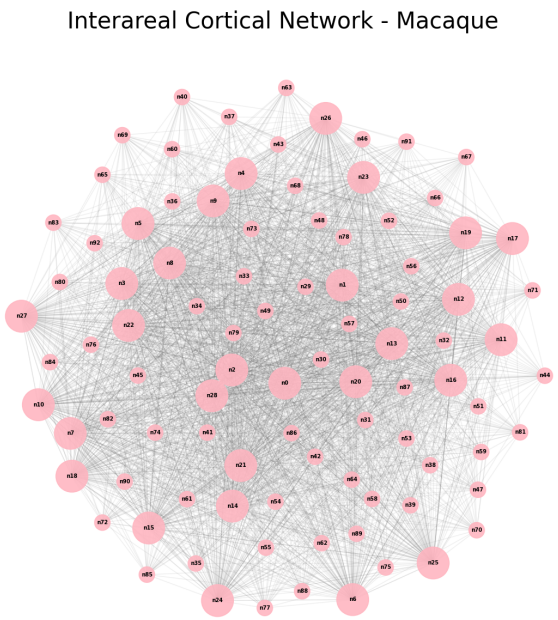

Figura 1: Rede Complexa Analizada

5 Resultados obitidos

Tabela 1: Análise Estrutural e de Ciclos

Total de Vértices	93
Total de Arestas	2667
Componentes Fracamente Conexas	1
Componentes Fortemente Conexas	65
Tamanho da Maior Comp. Forte	29 vértices (31.2% da rede)
Possui Ciclos?	Sim
Auto-loops	0
Ciclos de tamanho 3 'perfeitos'	0

Tabela 2: Top 5 Vértices - Métricas de Centralidade

Vértice	**Degree**	**Betweenness**	**Closeness**
n0	1.3043	0.0000	1.0000
n1	1.3043	0.0000	1.0000
n2	1.3043	0.0000	1.0000
n3	1.3043	0.0000	1.0000
n4	1.3043	0.0000	1.0000

Tabela 3: Distâncias

Parâmetro	**Resultado**
Tamanho	29 vértices
Raio	1
Diâmetro	2
Periferia	['n7']
Centro	['n11', 'n18', 'n26', 'n15', 'n8', 'n10', 'n22', 'n1', 'n6', 'n14', 'n4', 'n16', 'n17', 'n19', 'n25', 'n3', 'n5', 'n12', 'n28', 'n23', 'n24', 'n21', 'n20', 'n2', 'n0', 'n27', 'n13', 'n9']

Tabela 4: Graus (Entrada vs. Saída)

Estatística	**Grau de Entrada**	**Grau de Saída**
Mínimo	0	27
Máximo	92	29
Média	28.68	28.68
Frequência	0 (64 vérticces)	29 (64 vértices)

Tabela 5: AGM, Clusters e Cliques

Análise	Detalhe
AGM	92 arestas, Peso Total: 4156.60
Clusters	Coeficiente Médio: 0.8320
Cliques	Total: 64 | Maior Tamanho: 30

Tabela 6: Ciclos eulerianos e hamiltonianos

Análise	Detalhe
Possui ciclo euleriano fechado?	**Não**
Possui caminho euleriano aberto?	**Não**
É hamiltoniano?	**Não, pois o melhor caminho precisou de 133 passos para visitar 93 vértices.**

6 Considerações Finais

Com base na análise feita, a rede se classifica principalmente como uma Rede Livre de Escala (Scale-Free). Isso se justifica pela presença marcante de Hubs, o que é uma característica da distribuição de lei de potência e do mecanismo de ligação preferencial (**Rich-get-Richer**). Além disso, a rede apresenta propriedades de Pequeno-Mundo (**Small-World**), pois combina um alto coeficiente de agrupamento (0.83) com caminhos mínimos curtos (Diâmetro = 2) , diferenciando-se das redes puramente aleatórias.

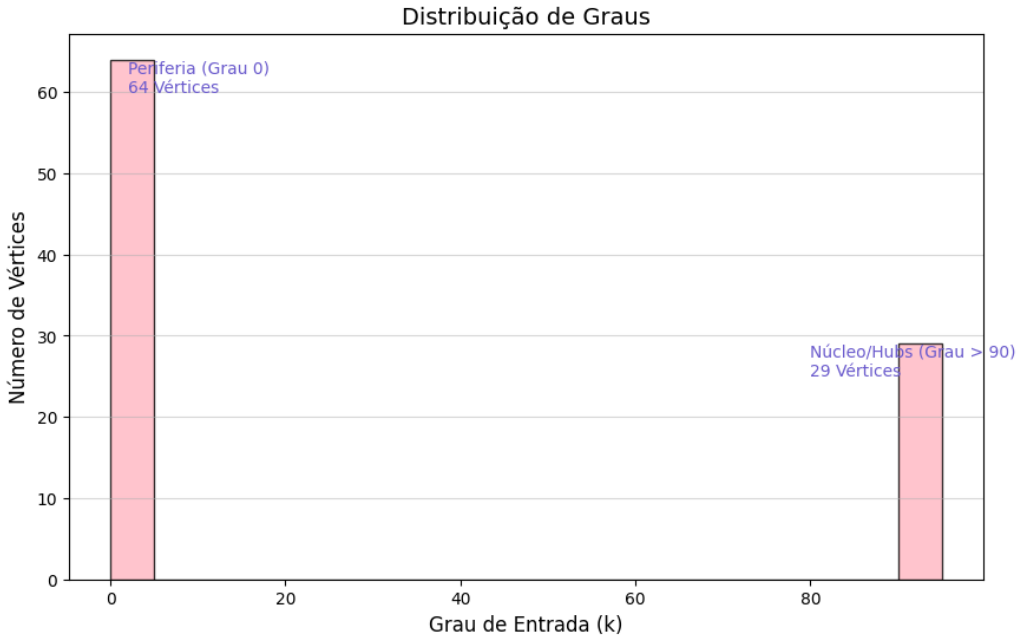

Figura 2: Distribuição de graus de entrada.

A Figura 2 revela uma distribuição de graus polarizada. Ao contrário de redes livres de escala comuns que possuem uma variação contínua de graus, essa rede apresenta uma separação exata: o grupo de vértices periféricos (grau 0) e um núcleo denso de hubs (grau > 90). Isso confirma visualmente a estrutura (**Rich-get-Richer**), onde o núcleo atua como um 'Rich-Club' que centraliza toda a conectividade da rede.